

Chapter 3

Server-Side Web Programming

1. MVC architecture in web development
2. Role of backend in web applications
3. Backend web framework: Django
4. Handling requests and responses
5. Form data handling and sessions
6. Routing, middleware, templating concepts
7. Overview and comparison of backend frameworks: Django, flask, fastapi, dot NET MVC framework, ruby on rails, java spring boot, node.js
8. Database integration: Relational versus NoSQL, CRUD operations, ORM concept
9. Authentication and authorization: Cookies, sessions, JWT
10. Middleware for logging, error handling, and security

Server-Side Web Programming or Backend development

Backend development refers to the server-side of a web application that works behind the scenes to process requests, manage data, apply business logic, and ensure security. The major significance of backend development is to manage data and ensure secure storage, retrieval and processing of the data.

Major Features of Backend Development

1. Request Handling & Routing
 - a. Receives HTTP requests (GET, POST, PUT, DELETE)
 - b. Routes requests to appropriate logic
 - c. Sends structured responses (JSON / HTML)
2. Business Logic Implementation
 - a. Enforces application rules
 - b. Controls workflows and validations
 - c. Example:
 - i. User must be 18+ to register
 - ii. Order cannot be placed without payment
 - iii. Stock must be available before checkout
 - d. Keeps logic centralized & consistent
3. Database Management
 - a. Stores, retrieves, updates, and deletes data
 - b. Ensures data integrity and relationships
4. Authentication & Authorization
 - a. Authentication
 - i. Verifies who the user is
 - ii. Uses credentials like username/password, tokens
 - b. Authorization
 - i. Controls what the user can access
5. Security
 - a. Protects application from attacks:
 - i. SQL Injection
 - ii. XSS
 - iii. CSRF
 - iv. Brute-force attacks
6. API Development & Integration
 - a. Backend exposes APIs for frontend & mobile apps
 - b. Allows third-party integrations
7. Session & State Management
 - a. Manages user sessions
 - b. Stores login state securely
 - c. Methods
 - i. Cookies
 - ii. Server-side sessions
 - iii. Token-based authentication
8. Error Handling & Logging

- a. Handles unexpected errors gracefully
 - b. Sends meaningful error responses
 - c. Logs errors for debugging
- 9. Performance & Scalability
 - a. Handles multiple users simultaneously
 - b. Optimizes response time
 - c. Techniques:
 - i. Caching (Redis)
 - ii. Load balancing
 - iii. Database indexing
 - iv. Async processing
- 10. Data Validation & Sanitization
 - a. Ensures correct data format
 - b. Prevents malicious input
- 11. File Handling
 - a. Upload & manage files
 - b. Store images, documents, videos
- 12. Communication Services
 - a. Email notifications
 - b. SMS alerts
 - c. Push notifications

Example:

Online Shopping App

Backend handles:

- User login
- Product storage
- Order processing
- Payment verification
- Admin control

Frontend : displays results.

**** Backend development is essential for handling *business logic, database operations, authentication, security, APIs, and scalability*, enabling web applications to function securely and efficiently.**

1. MVC architecture in web development

MVC architecture (Model–View–Controller) is a software design pattern used to separate an application into three interconnected parts, making it easier to develop, maintain, test, and scale—especially in web applications.

Model

Model represents the data layer and business logic of an application. Model defines the structure of data to be stored in the database and performs CRUD operations on the database.

What it does:

- Handles data and business logic
- Interacts with the database
- Performs CRUD operations

Examples:

- Database schemas
- ORM models (Sequelize, Prisma, Mongoose, Django Models)

View

In MVC, the View is responsible for:

- Presenting data to the user
- UI / layout / visual representation
- Showing data received from the Controller
- No business logic, no database logic

In **Traditional MVC View** (Server-Rendered):

In classic MVC (Django, Laravel, Express + EJS):

- Backend renders HTML
- View = template files
- Templates: HTML, EJS, Pug, Jinja, Blade

Modern architecture where backend serve APIs and Frontend handles all the presentation parts:

Also Called

- Decoupled MVC
- API-based architecture
- Client–Server separation

In such architecture, React is the view. Because:

1. Displays data (tables, cards, forms)
2. Handles user interaction (clicks, inputs)

3. Updates UI based on API responses
4. Does not manage DB or business rules

Controller

The Controller acts as the middleman between the Model and the View.

Controller = Request handler + Decision maker

Main Responsibilities of Controller

1. Handle User Requests
 - a. Receives input from the user
 - b. In web apps, this means HTTP requests
 - c. Examples:
 - i. GET /users
 - ii. POST /login
2. Coordinate with Model
 - a. Calls Model methods to:
 - b. Fetch data
 - c. Save data
 - d. Update data
 - e. Delete data
 - f. Does not contain business rules (ideally)
3. Return Response
 - a. Sends data to:
 - i. View (HTML) → traditional MVC
 - ii. OR JSON → API-based architecture

How MVC Works (Flow)

1. User sends a request (e.g., /users)
2. **Controller** receives the request
3. Controller asks **Model** for data
4. Model returns data
5. Controller sends data to **View**
6. View displays the result

Backend web framework: Django

Django is a high-level Python web framework used to build secure, scalable, and maintainable backend web applications quickly. It follows the MVT (Model–View–Template) architecture and promotes rapid development with clean and pragmatic design. Django helps developers focus on writing applications instead of reinventing common web functionalities.

Django uses **MVT (Model – View – Template)**

Component	Role
Model	Handles database structure and data
View	Handles business logic and request processing
Template	Handles UI (HTML response)

Django's View $\approx$ Controller in MVC.

Core Features of Django

1. ORM (Object Relational Mapping)
 - a. Interact with database using **Python classes**
 - b. No need to write SQL manually
2. Built-in Admin Panel
 - a. Automatic admin dashboard
 - b. Manage database records visually
3. URL Routing System
 - a. Maps URLs to views cleanly:

```
class Student(models.Model):  
  
    name = models.CharField(max_length=100)  
  
    age = models.IntegerField()
```

- c. Supports MySQL, PostgreSQL, SQLite, Oracle database
4. Authentication & Authorization
 - a. Built-in system for:
 - i. Login
 - ii. Logout
 - iii. Signup
 - iv. Password hashing
 - v. User roles & permissions
 - b. Secure by default
 - c. Saves huge development time
 5. Form Handling
 - a. Django forms handle:
 - i. Validation

```
path('students/', views.student_list)
```

- ii. CSRF protection
 - iii. Error handling
- 6. Security Features
 - a. Django protects against:
 - i. SQL Injection
 - ii. Cross-Site Scripting (XSS)
 - iii. Cross-Site Request Forgery (CSRF)
 - iv. Clickjacking
 - b. Ideal for production-level applications
- 7. Template Engine
 - a. Dynamic HTML rendering
 - b. Uses template tags and filters

```
{% for student in students %}
```

```
<p>{{ student.name }}</p>
```

```
{% endfor %}
```

- c. Separation of logic and UI
- 8. REST API Support (Django REST Framework – DRF)
 - a. Used to build **API-only backends** for:
 - i. React
 - ii. Next.js
 - iii. Mobile apps
 - b. Features
 - i. Serialization
 - ii. Authentication
 - iii. Pagination
 - iv. Permissions
 - c. Perfect for modern frontend + backend separation
- 9. Middleware Support
 - a. Middleware sits between:
 - b. Request → View → Response
 - c. Used for:
 - i. Authentication checks
 - ii. Logging
 - iii. Security
 - iv. Session handling
 - d. Scalability & Performance
 - i. Handles high traffic websites
 - ii. Supports caching
 - iii. Load balancing ready

Advantages of Django

1. Fast Development
 - Ready-made components
 - Less boilerplate code
2. Highly Secure
 - Security handled by framework
3. Scalable
 - From small apps to large platforms
4. Clean Code Structure
 - Easy to read
 - Easy to maintain
5. Large Community
 - Excellent documentation
 - Lots of third-party packages
6. Python Based
 - Easy for beginners
 - Powerful for professionals

Significance of Django:

1. Django is a Python backend web framework
2. Follows MVT architecture
3. Provides ORM, Admin panel, Security, Authentication
4. Promotes rapid and secure development
5. Best suited for scalable and production-ready applications.

Handling requests and responses

Client → HTTP Request → Middleware → Route Handler → Response → Client

In web applications:

1. Client (Browser / React / Mobile App) sends an HTTP request
2. Server processes the request
3. Server sends back an HTTP response

This is called the Request–Response Cycle.

Request–Response Handling in Node.js (Express)

Overview

- Node.js uses an **event-driven, non-blocking I/O model**
- Express.js simplifies request handling
- Each request is handled by a **route handler function**

Basic Express Example:

```
const express = require('express');  
  
const app = express();  
  
app.get('/hello', (req, res) => {  
  res.send('Hello from Node.js');  
});  
  
app.listen(3000);
```

Components of a request/response cycle:

A. Request Object (req)

The req object contains all information about the client request.

Common properties:

Property	Description
req.method	HTTP method (GET, POST, etc.)
req.url	Requested URL
req.params	URL parameters
req.query	Query string
req.body	Data sent by client

Eg:

```
app.get('/user/:id', (req, res) => {  
    console.log(req.params.id);  
});
```

B. Response Object (res)

The res object is used to send data back to the client.

Common methods:

Method	Purpose
res.send()	Send text / JSON
res.json()	Send JSON
res.status()	Set status code
res.redirect()	Redirect user

Example:

```
res.status(200).json({ message: "Success" });
```

C. Middleware in Express

Middleware are the functions that

- Run **before** the route handler
- Has access to all the req and res objects
- Can modify req and res objects
- Can execute logics and complete response cycle at any instance

Uses:

- Logging
- Authentication
- Parsing request body

D. Handling Different HTTP Methods

```
app.get('/login', (req, res) => {})  
app.post('/login', (req, res) => {})  
app.put('/user/:id', (req, res) => {})  
app.delete('/user/:id', (req, res) => {})
```

Request–Response Handling in Django

Overview:

- Django follows a synchronous request–response model
- Uses views to handle requests
- Built on WSGI/ASGI

WSGI and ASGI

WSGI and ASGI are connectors (or bridges). They help a web server talk to a Django application. A web server cannot directly understand Python code, so it needs a rule or standard to communicate.

That rule is:

- WSGI (older) Web Server Gateway Interface => Synchronous
- ASGI (newer) Asynchronous Server Gateway Interface => Asynchronous

How WSGI works (step by step)

1. User opens a website
2. Browser sends request (HTTP)
3. Web server receives it
4. Web server passes request to Django using **WSGI**
5. Django processes the request
6. Django sends response back using **WSGI**
7. Web server sends response to browser

ASGI was created because WSGI cannot handle:

- Chat applications
- Live notifications
- Real-time updates
- WSGI handles one request at a time. So, other requests must wait.

How ASGI works (step by step)

1. User sends request
2. Web server receives it
3. Web server sends it to Django using **ASGI**
Django can:
 - Work on many requests together
 - Pause one request and work on another
4. Response is sent back to browser

Flow in Django

Client → URLconf → View → Template / JsonResponse → Client

Basic Django View Example:

```
from django.http import HttpResponse
def hello(request):
    return HttpResponse("Hello from Django")
```

URL mapping:

```
path('hello/', hello)
```

Components of Django

A. Request Object (**request**)

The **request** object contains all request data.

Common attributes:

Attribute	Description
request.method	HTTP method
request.GET	Query parameters
request.POST	Form data
request.body	Raw data
request.headers	Request headers
request.user	Logged-in user

Example:

```
def user_view(request):
    user_id = request.GET.get('id')
```

B. Response Object in Django

Django provides different response classes:

Response Type	Purpose
HttpResponse	Plain text
JsonResponse	JSON data
HttpResponseRedirect	Redirect
render()	HTML template

Eg:

```
from django.http import JsonResponse

def api(request):

    return JsonResponse({"message": "Success"})
```

C. Handling HTTP Methods in Django

```
def login(request):

    if request.method == "POST":

        return HttpResponseRedirect("Login Success")

    return HttpResponseRedirect("Invalid Request")
```

D. Middleware in Django

Middleware runs globally for every request.

Uses:

- Authentication
- Security
- Sessions
- Logging

Node.js vs Django

Aspect	Node.js (Express)	Django
Language	JavaScript	Python
Architecture	Event-driven	MVT
Request Object	req	request
Response Object	res	HttpResponse
Routing	Inside app	Separate URL config
Middleware	Function-based	Class-based
Async Support	Native	Via ASGI
API Development	Lightweight	Structured

Form data handling and sessions

Form data is the information sent by the client (browser) to the server when a user submits a form.

Eg: login forms, registration forms, feedback forms, data entry forms, etc.

Routing, middleware, templating concepts

Routing

Routing is the process of mapping a URL request to a view function that handles the request.

In Django, routing is handled using URL Configuration (URLconf).

Middleware

Middleware is a component that processes requests before they reach the view and processes responses before they are sent back to the client.

Middleware works globally.

Templating

Templating is the process of generating dynamic HTML pages by combining:

- HTML
- Data from view

Django uses Django Template Language (DTL).

